

Sharing `*terminal-io*` (and the standard streams) between threads in SBCL and CCL — applied to aldo

June 24, 2026

Contents

- **Status:** design note / research input for aldo (the clautolisp AutoLISP debugger).
- **Scope:** SBCL and Clozure CL (CCL); dumb-terminal, ncurses, and salve (TCP) debugger front-ends.

1. The aldo model, restated

aldo runs the debugger in a **separate thread** from the AutoLISP program:

- The **main thread** evaluates AutoLISP, does I/O on the standard streams, and computes.
- On a debugger event (`invoke-debugger`, `break`, a breakpoint hit, a step), the main thread **parks itself** and **wakes the debugger thread**, which then does introspection and user interaction.
- Control is handed back and forth; at any instant **only one** of the two threads is using the terminal.

The central assumption: *because execution is exclusive, the mutex on `*terminal-io*` is effectively ensured, and threads only swap the terminal between completed Common Lisp I/O operations (not necessarily between AutoLISP I/O operations — we may be mid-way through printing a list).*

That assumption is **correct and sufficient on CCL by construction, and correct on SBCL only if you add the missing pieces yourself**. The two implementations make very different promises about shared streams, and they lay out the standard streams differently. The rest of this note documents exactly what each one guarantees and what also has to do.

The short version:

Concern	CCL (Clozure)
Per-operation locking on built-in streams	Yes , automatic
terminal-io shared across all threads	Yes , one shared stream
Terminal- <i>input</i> ownership protocol	Yes : WITH-TERMINAL-INPUT
standard-output vs *terminal-io* backing	Same underlying stream (ANSI synonym layout)
User-created shared stream default	:sharing :private (owner-only, errors x-thread)

2. Clozure CL (CCL)

2.1 The **:sharing** attribute

CCL attaches a *sharing mode* to every stream. **OPEN** and **MAKE-SOCKET** take a **:SHARING** keyword with three documented values (CCL manual §10.1.2):

- **:private** — the stream may be used only by the thread that *first* does I/O on it; that thread becomes the "owner." I/O from any other thread **signals an error**. This is the default for **OPEN~/~MAKE-SOCKET**. (Note: ownership is acquired on first I/O, not at creation, and there is no implemented "transfer ownership" operation.)
- **:lock** — every access takes a lock. I/O streams have **separate read and write locks**, so one thread can be reading while another writes. This is the mode you want for a stream genuinely shared by several threads.
- **:external** (**nil** is a synonym) — no access protocol at all; the application is responsible. CCL warns that even *reading* mutates internal stream state, so concurrent unsynchronised access corrupts the stream.

Source: <https://ccl.clozure.com/manual/chapter10.1.html>

Actionable: any stream *aldo itself creates* and shares between the main and debugger threads (e.g. a socket for *salve* used by both a reader and a

writer thread, or an fd-stream you build for the terminal) must be created with `:sharing :lock`. The default `:private` will throw the moment the second thread touches it. If aldo manages its own application-level handoff and wants to avoid even the lock overhead, `:sharing :external` is available — but then *all* mutual exclusion is on you.

2.2 `*terminal-io*` is already shared and already locked

This is the important part, and it maps almost one-to-one onto aldo. From CCL manual §7.5 (“Background Terminal Input”, <https://ccl.closure.com/manual/chapter7.5.html>):

- All Lisp processes (threads) **share one common `*terminal-io*`**, and therefore share `*debug-io*`, `*query-io*`, and the other standard interactive streams derived from it.
- **All I/O to CCL’s buffered streams goes through a locking mechanism.** Output from a background thread to `*terminal-io*` is therefore safe; the worst case is cosmetic interleaving, not corruption. So the standard interactive streams behave as if `:sharing :lock`, **not `:private`** — cross-thread output does not error.

So for **output**, aldo on CCL needs to do nothing special: the main thread and the debugger thread can both write to `*terminal-io*` / `*standard-output*` and CCL serialises the buffer manipulation for you. Your exclusive-handoff design makes even the cosmetic interleaving a non-issue, because the two threads are never actually writing at the same time.

2.3 Terminal *input* uses an ownership protocol — use it

Input is the genuinely hard direction, and CCL already solves exactly aldo’s problem:

- The internal stream that supplies terminal input is **always owned by some thread**. The initial thread (the one running the top-level REPL) owns it at startup.
- A background thread that needs to read from the terminal calls `WITH-TERMINAL-INPUT`, which *requests* ownership and *promises to hand it back* to the previous owner on exit. A background thread that just calls `READ~/~READ-CHAR` without this protocol will typically **hang forever**.

- Crucially: in CCL, `BREAK`, `ERROR`, `CERROR`, `Y-OR-N-P`, `YES-OR-NO-P`, `CCL:GET-STRING-FROM-USER`, and the `:TTY` interfaces to `STEP` and `INSPECT` are all already implemented on top of `WITH-TERMINAL-INPUT`.

That last point means aldo’s dumb-terminal UI, when it enters interaction from the debugger thread, should wrap its terminal reads in `WITH-TERMINAL-INPUT` (the same way CCL’s own `:TTY` stepper does) so that ownership is correctly transferred from the parked main thread and restored afterward. If you build aldo’s break entry on top of the standard condition machinery, you largely get this for free.

CCL is candid that this scheme is not bulletproof when *many* threads contend for input simultaneously, and the long-term “right” fix it suggests is giving each thread its own `*terminal-io*` via network or window-system streams — which is precisely what your salve back-end does (see §5.3).

3. SBCL

SBCL makes the opposite design choice on both counts: a different stream layout, and **no automatic locking**.

3.1 Stream layout — and why it bites the “same stream” assumption

SBCL sets up (as described by Nikodemus Siivola and documented in Verna’s CDR-11 stream study, <https://cdr.common-lisp.dev/document/11/verna.12.cdr11.html>):

- Four underlying streams: `*stdin*`, `*stdout*`, `*stderr*`, and `*tty*`.
- `*tty*` is an fd-stream on `/dev/tty` if that can be opened; otherwise it falls back to `(make-two-way-stream *stdin* *stdout*)`.
- `*standard-input*`, `*standard-output*`, `*error-output*` start as **synonym streams** to `*stdin*`, `*stdout*`, `*stderr*` respectively.
- `*terminal-io*`, `*query-io*`, `*debug-io*` start as **synonym streams** to `*tty*`.

This deliberately deviates from the CLtL2/ANSI default layout (where the standard streams are synonyms of `*terminal-io*`, CLtL2 §21.1). SBCL does it so that **shell redirection of stdin/stdout is honoured** by `*standard-output*/~/*error-output*` while the interactive streams stay glued to the controlling terminal.

Consequence for aldo — **read this twice.** On SBCL, when an interactive terminal is present:

- The debugged AutoLISP program’s `*standard-output*` flows to `*stdout*` → `fd 1`.
- aldo’s dumb-terminal UI writing to `*terminal-io*` flows to `*tty*` → `/dev/tty`, a *different file descriptor* with its *own buffer*.

So the premise “the dumb terminal and the debuggee share the same `*terminal-io*` stream” is **false on SBCL by default**. They land on the same physical device but through two distinct fd-streams. Your mutual-exclusion handoff prevents *concurrent* access, but it does **not** prevent *buffered-but-unflushed* output on one stream from appearing after text written later to the other. You must therefore **finish-output** the debuggee’s stream(s) before handing control to the debugger, and **finish-output** the debugger’s stream before resuming — regardless of the mutex.

If you want the simple “one shared stream” mental model on SBCL, the cleanest fix is to make aldo’s dumb-terminal UI write to **the same stream the debuggee uses** (e.g. bind both to one fd-stream, or have the debugger use `*standard-output*`~/`*standard-input*` rather than `*terminal-io*`), so there is exactly one buffer and one fd to reason about.

3.2 SBCL does not lock streams for you

SBCL’s fd-streams are **not synchronised** for concurrent access. There is no `:sharing` analogue and no per-operation lock on the built-in streams; concurrent operations on the same buffered fd-stream can corrupt the buffer state (SBCL’s own porting notes put it bluntly: for buffered fd-streams, simultaneous use will be mishandled). The implementation provides the mutex primitives (`sb-thread:make-mutex`, `sb-thread:with-mutex`, or `bordeaux-threads`) and expects **you** to wrap shared-stream access.

This is why SLIME/Swank does not hand a raw fd-stream to multiple threads: it installs its own Gray stream objects for the REPL and serialises access at that layer. That is the canonical pattern to imitate on SBCL.

Actionable for aldo on SBCL:

1. Hold an explicit **aldo** mutex around any stream touched by both threads — even though your handoff already serialises in time, the mutex is your portable correctness statement and a backstop should any access ever escape the synchronous entry path (§6).

2. Flush (**finish-output**) at every handoff boundary, because of the stdout-vs-tty buffer split.
3. Your "swap only between completed CL I/O operations" rule is exactly the right granularity: SBCL corruption comes from *interleaving inside one operation's buffer manipulation*, and you have ruled that out for the synchronous (breakpoint/step) path.

4. Side-by-side, for the dumb-terminal case

The dumb-terminal UI is the only case where the debugger and debuggee contend for the *same physical stream*. Putting the two implementations together:

- **CCL.** The shared ***terminal-io*** is locked automatically (output is safe; input goes through **WITH-TERMINAL-INPUT** ownership). With ***standard-output*** as the usual ANSI synonym of ***terminal-io***, both threads drive one underlying buffered stream, so a single lock — CCL's — already covers everything. aldo's job is mainly to use **WITH-TERMINAL-INPUT** (directly or via the standard **break~/~step** machinery) when the debugger thread reads.
- **SBCL.** No automatic lock; ***terminal-io*** (*dev/tty*) and ***standard-output*** (*stdout*) are usually /different fd-streams. aldo must (a) supply its own mutex, (b) flush at handoffs, and ideally (c) collapse the two onto a single stream so the "shared ***terminal-io***" model actually holds.

Because your design enforces temporal exclusivity, neither implementation will *interleave* the two threads in practice. The residual differences are: who owns the lock (CCL: the runtime; SBCL: aldo), and whether the debugger and debuggee share one buffer (CCL: yes by default; SBCL: no by default).

5. Applying it to the three front-ends

5.1 Dumb terminal (***terminal-io***)

Covered above. Summary of what aldo should do:

- **CCL:** rely on built-in locking for output; enter debugger input via **WITH-TERMINAL-INPUT** (or via standard **break~/~invoke-debugger**, which already use it). Nothing else required.

- **SBCL:** one `aldo` mutex around terminal access; `finish-output` on handoff; prefer binding the debugger UI to the same stream object the debuggee uses, so there is a single fd and buffer.

5.2 ncurses (terminal in 2D mode)

Your instinct is right: drop below the Lisp stream layer and talk to the terminal fd directly, then save and restore screen state so the intervention is transparent to the debuggee. Concretely:

- **Terminal mode.** Save the current `termios` (`tcgetattr`) before switching to raw/cbreak mode for curses, and restore it (`tcsetattr`) on exit. `ncurses` wraps this as `def_prog_mode` / `def_shell_mode` + `savetty` / `resetty`, with `endwin` to leave curses mode.
- **Screen contents.** Use the terminal's **alternate screen buffer** so the debuggee's scrollbar is untouched: terminfo capabilities `smcup` (enter, "ca mode") / `rmcup` (exit), which on xterm-family terminals are `ESC [ ? 1049 h` / `ESC [ ? 1049 l`. Enter on debugger activation, exit on resume; the debuggee's screen reappears exactly as it was.
- **Buffer discipline.** Because curses writes the terminal fd directly while the debuggee writes through a CL fd-stream to (often) the same fd, you must **finish-output the debuggee's output stream(s) before entering curses mode**, and again after `endwin`, so half-buffered Lisp output cannot land on top of the curses screen (or vice versa). The temporal exclusivity of your handoff guarantees the two never write concurrently; the flush guarantees nothing is *pending* across the boundary.
- **Implementation-independence.** Neither CCL's stream lock nor SBCL's lack of one matters much here, since curses bypasses the Lisp stream entirely. The one cross-implementation wrinkle is obtaining the actual terminal fd: on CCL/SBCL alike you can go through the FFI to the controlling terminal (`/dev/tty`) rather than assuming fd 0/1, which also sidesteps SBCL's stdout-vs-tty split cleanly (curses and SBCL's `*tty*` then agree on `/dev/tty`).

5.3 salve protocol (TCP/IP to aldb in Emacs, local or remote)

This is the easy and the *recommended* case, and it is what CCL's manual itself points to as the clean long-term answer to terminal sharing: give the

debugger its own stream rather than multiplexing the console.

- The debugger I/O is on **socket streams entirely separate** from the debuggee's standard streams, so there is **no contention with the debuggee's terminal/stdout at all**. The debuggee keeps writing to its own console; aldo talks to aldb over the socket.
- The only synchronisation question is *internal to aldo*: if a reader thread and a writer thread share the one socket stream, protect it.
 - **CCL**: create the socket with `:sharing :lock` — you then get independent read/write locks, so the reader and writer don't block each other. Do **not** leave it at the default `:sharing :private`, or the second thread will get an error.
 - **SBCL**: wrap shared socket access in your own mutex (or, as usual, dedicate one direction to one thread).
- Remote vs local makes no difference to the stream model; it only changes latency and framing.
- This back-end also dissolves the SBCL stdout-vs-tty issue and the CCL terminal-ownership dance entirely, because the debugger never touches `*terminal-io*`. For multi-front-end work, salve is the design that scales.

6. Asynchronous entry, made synchronous by deferral

Everything above assumes the **synchronous** path: the main thread reaches a breakpoint/step check *between* completed CL I/O operations, signals the debugger, and parks itself. At that point buffers are in a consistent state and the handoff is clean. This is the right design and it is safe on both implementations.

The hazard is **asynchronous** debugger entry — a user interrupt (Ctrl-C / SIGINT), a watchdog `process-interrupt` (CCL) / `sb-thread:interrupt-thread` (SBCL), or `invoke-debugger` from an arbitrary dynamic context. Naïvely, the interrupted thread could be *mid-buffer manipulation* on a stream, and entering the debugger right there would risk (CCL) blocking on — or deadlocking against — a stream write lock the interrupted thread still holds, or (SBCL) writing through a buffer the interrupt left half-updated.

6.1 The design: interrupts only arm a flag; entry happens at the next poll point

aldo removes the hazard entirely by **never entering the debugger from the interrupt itself**. When `process-interrupt` / `interrupt-thread` (or any async trigger) fires, the handler does the minimum: it **sets a "pending break" flag** in the main thread and returns. The interrupted code resumes and runs to its **next poll point** — the same per-AutoLISP-operation check aldo already uses for breakpoints and stepping — where it observes the flag and enters the debugger *exactly as if a breakpoint had been set at that point*.

The effect is that **all** debugger entry — breakpoints, stepping, and now interrupts — flows through one synchronous code path, at CL-operation granularity, with consistent buffers and no lock held across the handoff. The async/sync distinction disappears from the debugger's point of view:

- **CCL:** there is no stream lock to deadlock on, because the interrupted primitive completes and releases any lock before the poll point is reached; the debugger thread then acquires the terminal via `WITH-TERMINAL-INPUT` from a cleanly parked main thread.
- **SBCL:** there is no half-updated buffer to corrupt, because the interrupted operation finishes before the flag is acted on; the usual handoff `finish-output` then applies.

The interrupt handler must stay trivial — set a flag (a ~special~/atomic boolean) and return. No I/O, no condition-system entry, no nontrivial allocation from the handler. That is async-signal hygiene anyway, and a single flag store satisfies it on both implementations.

6.2 The residual tradeoff: responsiveness vs. blocking points

Deferral trades immediacy for consistency, and that tradeoff is worth stating explicitly:

- **Latency to break is bounded by poll-point density.** The interrupt takes effect at the next check, not instantly. For ordinary compute and AutoLISP-op loops this is imperceptible; denser poll points (loop back-edges, function entry, op boundaries) make it tighter at some runtime cost.
- **A thread blocked with no reachable poll point won't break promptly.** If the main thread is parked in a blocking primitive —

`read` waiting on input, `sleep`, or an FFI call into the CAD host that doesn't return — it reaches no poll point until that call returns, so the pending break waits. This is the one case where "Ctrl-C feels stuck." If aldo needs to break *into* a blocking read (e.g. cancel a hung `getstring`), that specific primitive must be made interrupt-aware: either use the implementation's interruptible/timeout-bearing I/O so the blocking call itself unwinds and reaches the poll point, or poll the flag inside aldo's own input loop rather than sitting in a bare blocking `read`. Decide per primitive whether prompt interruptibility is worth that; for most, deferral to the next op boundary is exactly right.

- **error / invoke-debugger raised by the program** is already synchronous — it unwinds through the condition system to a definite evaluation boundary, not mid-buffer — so it is safe to enter directly. For uniformity, route it through the same entry path (e.g. have the handler arm the flag and unwind to the nearest poll/handler frame) so there is a single place that performs user I/O.

On the **salve** back-end this is all moot regardless: the debugger's socket is unrelated to whatever buffer the main thread was touching, so even immediate entry wouldn't corrupt the debuggee's streams. The deferral design is still worth keeping there for a different reason — it gives one uniform entry path and consistent, snapshot-at-a-known-point program state across all three front-ends.

7. Recommendations for aldo's portability layer

A thin internal abstraction keeps the implementation differences out of the debugger logic:

1. `with-debugger-terminal ((stream) &body body)`

- CCL dumb-terminal: expands to `WITH-TERMINAL-INPUT` around `body`.
- SBCL dumb-terminal: expands to `(sb-thread:with-mutex (*aldo-terminal-lock*) ... )` with a `finish-output` on entry/exit.
- ncurses (both): save `termios` + enter alt-screen on entry, restore + `endwin` on exit, with debuggee-stream flushes around it.
- salve (both): no-op with respect to the terminal (the socket is the channel).

2. **One synchronous entry path; interrupts only arm a flag.** Async triggers (`process-interrupt` / `interrupt-thread`, Ctrl-C, out-of-context `invoke-debugger`) set a "pending break" flag and return; entry happens at the next poll point, just like a breakpoint (§6). This makes buffer state consistent and the handoff lock-free on both implementations, at the cost of poll-point-bounded latency. Keep the handler trivial (flag store only). For primitives where you need to break into a *blocking* call, make that primitive interrupt-aware rather than entering from the signal.
3. **Create every aldo-shared stream explicitly shareable.** On CCL pass `:sharing :lock`; never inherit the `:private` default for a stream two threads will touch. On SBCL there is nothing to pass — instead guarantee a mutex wraps it.
4. **Always flush at handoff boundaries.** Cheap, and it papers over both the SBCL stdout/tty split and any buffering surprise on either implementation.
5. **Prefer salve as the primary architecture.** It is the only back-end with *no* shared-stream problem, it is what CCL's own documentation recommends as the clean solution, and it is the one that generalises to remote and multiple front-ends.
6. **Verify, don't assume, the runtime layout.** At startup aldo can check, per implementation, whether `*standard-output*` and `*terminal-io*` resolve to the same underlying stream/fd (they usually do on CCL, usually don't on SBCL with a tty) and adapt — e.g. by collapsing them for the dumb-terminal back-end.

8. References

- CCL manual §10.1, "Stream Extensions" (the `:SHARING` keyword: `:private` / `:lock` / `:external`): <https://ccl.clozure.com/manual/chapter10.1.html>
- CCL manual §7.5, "Background Terminal Input" (shared `*terminal-io*`, automatic buffered-stream locking, the `WITH-TERMINAL-INPUT` ownership protocol, and the list of standard operators built on it): <https://ccl.clozure.com/manual/chapter7.5.html>

- CCL streams source (sharing semantics, ioblock locking): <https://github.com/Clozure/ccl/blob/master/doc/manual/streams.ccldoc>
- D. Verna, "Standard io-syntax / standard stream behaviour across implementations," CDR-11 — SBCL's `*stdin*` / `~::~*stdout*` / `~::~*stderr*` / `~::~*tty*` layout and synonym-stream bindings, and the rationale (honouring shell redirection): <https://cdr.common-lisp.dev/document/11/verna.12.cdr11.html>
- CLtL2 §21.1, "Standard Streams" (the ANSI default: standard streams as synonyms of `*terminal-io*`): <https://www.cs.cmu.edu/Groups/AI/html/cltl2/clm/node183.html>
- SBCL manual, Threading chapter (mutex primitives; no claim of automatic stream synchronisation): <https://www.sbcl.org/manual/> · source: <https://github.com/sbcl/sbcl/blob/master/doc/manual/threading.texinfo>
- SBCL fd-stream source (buffered fd-stream internals; no per-operation lock): <https://github.com/sbcl/sbcl/blob/master/src/code/fd-stream.lisp>